KỸ THUẬT LẬP TRÌNH

BÀI TẬP LỚN

Nhóm 2–3 sinh viên | Năm học 2025–2026

YÊU CẦU CHUNG CHO TẤT CẢ ĐỀ TÀI

1. Phương pháp thiết kế (bắt buộc)

- Phân tích đề → xác định dữ liệu/thao tác → phác main() pipeline → tách hàm → kiểm thử
- Nộp kèm tài liệu thiết kế: sơ đồ hàm, mô tả input/output từng hàm (1–2 trang)

2. Struct lồng nhau (bắt buộc mọi đề)

- Mỗi đề có ít nhất 2 cấp struct lồng nhau (struct chứa vector<struct khác>)
- Hàm thao tác phải nhận/trả struct cấp ngoài; không truy cập trực tiếp field cấp trong từ main()

3. Dãy con / Mảng con (bắt buộc mọi đề)

- Mỗi đề phải áp dụng ít nhất 2 trong số: Kadane, LIS (tư duy), Sliding window, Prefix sum
- Thao tác trên mảng/dãy phải là hàm độc lập, nhận vector<T> làm tham số

4. Clean code & Tổ chức module

- Đặt tên rõ, mỗi hàm một nhiệm vụ, DRY, không lồng sâu quá 3 cấp
- Tách tối thiểu 3 file: io.h/.cpp – processing.h/.cpp – main.cpp

5. Kiểm thử

- ≥3 bộ test từ file: test thường + case biên + case dữ liệu lớn
- Mỗi bộ test có expected output, nộp kèm file test và README build/run

BTL01 *[Khí tượng & Môi trường]*

Hệ thống phân tích dữ liệu thời tiết nhiều trạm đo

Mô tả bài toán

Mỗi trạm đo lưu nhiều bản ghi đo (ngày, nhiệt độ, độ ẩm, lượng mưa). Chương trình đọc dữ liệu nhiều trạm từ CSV, thống kê theo từng trạm, tìm đoạn ngày bất thường và xuất báo cáo tổng hợp.

Thiết kế Struct lồng nhau

```
struct Record  { string date; double temp, humidity, rain; bool valid; }
struct Station { string id, name; vector<Record> records; Stats stats; }
→ Station chứa mảng Record: struct lồng nhau cấp 2
```

Input / Output

- Input: stations.csv (station_id, name, date, temp, humidity, rain — có missing/NA)
- Output: station_report.csv (thống kê từng trạm), anomaly.txt (đoạn bất thường)

Yêu cầu kỹ thuật

Kỹ thuật	Yêu cầu cụ thể
Struct lồng nhau	Station chứa vector<Record>; Stats lồng trong Station; đọc/ghi toàn bộ qua hàm io
File/CSV	Đọc nhiều dòng vào vector<Station>, xử lý missing, validate từng Record
Dãy con – Kadane	Trên mảng nhiệt độ của mỗi trạm: tìm đoạn ngày có tổng nhiệt độ cao nhất
Dãy con – tăng dài nhất	Tìm chuỗi ngày liên tiếp lượng mưa tăng dần (LIS tư duy trên records[])
Prefix sum 1D	Trên mảng lượng mưa của trạm: trả lời truy vấn tổng đoạn [l,r] O(1)
Thống kê mô tả	mean/std/min/max theo từng trạm; chuẩn hoá Z-score nhiệt độ
Xử lý chuỗi	Chuẩn hoá tên trạm, parse ngày, phân loại mức mưa theo chuỗi
Tổ chức module	io / station_processing / analytics / report — tối thiểu 3 file .h/.cpp
M0 + Test	≥3 bộ test; case biên: trạm 0 bản ghi, toàn missing, 1 ngày đo

BTL02 *[Tài chính & Chứng khoán]*

Hệ thống phân tích danh mục cổ phiếu

Mô tả bài toán

Mỗi cổ phiếu có chuỗi phiên giao dịch (ngày, giá đóng cửa, khối lượng). Chương trình quản lý danh mục nhiều cổ phiếu, tính MA-k, tìm giai đoạn lợi nhuận tốt nhất và tối ưu cửa sổ thời gian đạt mục tiêu.

Thiết kế Struct lồng nhau

```
struct Session  { string date; double close; long volume; double ma; }
struct Stock    { string ticker; vector<Session> sessions; PortfolioStats stats;
}
struct Portfolio{ string name; vector<Stock> stocks; }
→ Portfolio → Stock → Session: struct lồng nhau cấp 3
```

Input / Output

- Input: portfolio.csv (ticker, date, close, volume); config.txt (k cho MA, target_gain S)
- Output: analysis.csv (giá + MA-k từng cổ phiếu), signals.txt, best_period.txt

Yêu cầu kỹ thuật

Kỹ thuật	Yêu cầu cụ thể
Struct lồng nhau	Portfolio chứa vector<Stock>, Stock chứa vector<Session>; hàm tách rõ từng tầng
File/CSV	Đọc và nhóm dữ liệu vào đúng Stock theo ticker, xử lý missing giá
Dãy con – Kadane	Trên mảng diff giá của mỗi Stock: tìm đoạn thời gian lợi nhuận cao nhất
Dãy con – tăng dài nhất	Tìm chuỗi phiên giá tăng liên tiếp dài nhất cho mỗi cổ phiếu
Sliding window	Tìm cửa sổ ngày ngắn nhất có tổng lợi nhuận ≥ S (dãy không âm)
Rolling mean (MA-k)	Cập nhật MA-k trên sessions[] bằng cửa sổ trượt; lưu vào Session.ma
Prefix sum 1D	Prefix sum trên close[] của mỗi Stock: truy vấn tổng giá đoạn [l,r] O(1)
Thống kê	mean/std/min/max giá theo từng Stock; chuẩn hoá Min-Max toàn danh mục
M0 + Test	≥3 bộ test; case biên: 1 phiên, toàn giảm, ticker không có dữ liệu

BTL03 *[Giáo dục & Học vụ]*

Hệ thống quản lý điểm và phân tích học lực sinh viên

Mô tả bài toán

Mỗi sinh viên có danh sách điểm nhiều môn học qua nhiều kỳ. Chương trình tính GPA, phân loại học lực, tìm đoạn kỳ học tiến bộ liên tiếp và phát hiện sinh viên có xu hướng sụt giảm.

Thiết kế Struct lồng nhau

```
struct Grade    { string subject; double score; int credit; string semester; }
struct Student  { string id, name; vector<Grade> grades; double gpa; string
rank; }
struct Cohort   { string class_id; vector<Student> students; }
→ Cohort → Student → Grade: struct lồng nhau cấp 3
```

Input / Output

- Input: grades.csv (student_id, name, subject, score, credit, semester); class.txt
- Output: gpa_report.csv, progress.txt (đoạn tiến bộ), ranking.csv

Yêu cầu kỹ thuật

Kỹ thuật	Yêu cầu cụ thể
Struct lồng nhau	Cohort chứa vector<Student>, Student chứa vector<Grade>; GPA tính từ grades[]
File/CSV	Đọc và nhóm Grade vào đúng Student; validate điểm [0,10], tín chỉ > 0
Dãy con – Kadane	Trên mảng GPA từng kỳ của SV: tìm đoạn kỳ học có tổng GPA cao nhất
Dãy con – tăng dài nhất	Tìm chuỗi kỳ học GPA tăng liên tiếp dài nhất (phát hiện tiến bộ bền vững)
Mảng con – Sliding window	Cửa sổ k kỳ: tính GPA trung bình trượt, phát hiện đoạn sụt giảm
Prefix sum 1D	Prefix sum tín chỉ tích lũy theo kỳ: truy vấn tổng tín chỉ đoạn [l,r] O(1)
Thống kê mô tả	mean/std/min/max điểm theo môn, theo lớp; chuẩn hoá Z-score
Xử lý chuỗi	Chuẩn hoá tên SV (trim, title case), phân loại học lực theo chuỗi rank
M0 + Test	≥3 bộ test; case biên: SV 0 môn, điểm toàn 0, 1 kỳ học

BTL04 *[Vật lý & Mô phỏng số]*

Thư viện đa thức hệ số phân số và mô phỏng quỹ đạo

Mô tả bài toán

Biểu diễn đa thức với hệ số là phân số chính xác (Fraction). Chương trình đọc danh sách đa thức từ CSV, tính giá trị tại nhiều điểm, tính đạo hàm, tìm đoạn x có P(x) cực đại và so sánh các đa thức.

Thiết kế Struct lồng nhau

```
struct Fraction    { int num, den; }  // hệ số chính xác
struct Polynomial  { string id; int degree; vector<Fraction> coeffs; }
struct Experiment  { string name; vector<Polynomial> polys; vector<double>
x_vals; }
→ Experiment → Polynomial → vector<Fraction>: struct lồng nhau cấp 3
```

Input / Output

- Input: polynomials.csv (id, degree, num0/den0, num1/den1, ...); x_values.csv
- Output: poly_report.csv (P(x), P'(x)); extrema.txt (đoạn x cực trị); compare.csv

Yêu cầu kỹ thuật

Kỹ thuật	Yêu cầu cụ thể
Struct lồng nhau	Polynomial chứa vector<Fraction>; Experiment chứa vector<Polynomial>; hàm thao tác tách rõ
Phân số (Fraction)	Struct Fraction: chuẩn hoá gcd, mẫu dương; cộng/nhân/so sánh phân số
Đa thức (Polynomial)	Tính P(x) bằng Horner; tính P'(x); cộng 2 đa thức (hệ số Fraction)
File/CSV	Đọc hệ số dạng 'num/den' hoặc số nguyên; xử lý missing hệ số → 0/1
Dãy con – Kadane	Trên mảng P(xi): tìm đoạn x liên tiếp có tổng P(x) lớn nhất
Dãy con – tăng dài nhất	Tìm chuỗi xi liên tiếp mà P(xi) tăng dần (LIS tư duy)
Prefix sum 1D	Prefix sum trên mảng P(xi): truy vấn tổng đoạn [l,r] O(1)
Thống kê	mean/std của dãy P(xi); chuẩn hoá Z-score; so sánh 2 đa thức theo mean
M0 + Test	≥3 bộ test; case biên: bậc 0, hệ số âm, mẫu = 0 (lỗi), x = 0

BTL05 *[Sinh học & Y tế]*

Hệ thống theo dõi chỉ số sức khoẻ bệnh nhân theo thời gian

Mô tả bài toán

Mỗi bệnh nhân có chuỗi lần đo chỉ số (ngày, glucose, huyết áp, BMI). Chương trình theo dõi xu hướng, phát hiện đoạn bất thường, tính rolling mean và xuất báo cáo nhóm nguy cơ.

Thiết kế Struct lồng nhau

```
struct Measurement { string date; double glucose, bp, bmi; bool valid; }
struct Patient     { string id, name; int age; vector<Measurement> records;
string risk; }
struct Ward        { string ward_id; vector<Patient> patients; }
→ Ward → Patient → Measurement: struct lồng nhau cấp 3
```

Input / Output

- Input: patients.csv (ward, patient_id, name, age, date, glucose, bp, bmi — có missing)
- Output: risk_report.csv (nhóm nguy cơ), trend.txt (đoạn bất thường), ward_summary.txt

Yêu cầu kỹ thuật

Kỹ thuật	Yêu cầu cụ thể
Struct lồng nhau	Ward chứa vector<Patient>, Patient chứa vector<Measurement>; stats tính từ records[]
File/CSV	Đọc và nhóm Measurement vào đúng Patient, validate (glucose>0, bp>0), xử lý missing
Dãy con – Kadane	Trên mảng glucose của Patient: tìm đoạn ngày có tổng glucose cao nhất (nguy cơ cao)
Dãy con – tăng dài nhất	Tìm chuỗi ngày liên tiếp glucose tăng dần (phát hiện xu hướng xấu)
Sliding window – rolling mean	Cửa sổ 7 ngày trên records[]: rolling mean glucose và bp; lưu vào output
Prefix sum 1D	Prefix sum trên mảng bp: truy vấn tổng huyết áp đoạn [l,r] O(1)
Thống kê mô tả	mean/std/min/max theo từng Patient và theo Ward; chuẩn hoá Min-Max BMI
Xử lý chuỗi	Chuẩn hoá tên bệnh nhân; phân loại nhóm nguy cơ theo chuỗi (low/medium/high)
M0 + Test	≥3 bộ test; case biên: bệnh nhân 0 lần đo, toàn missing, chỉ số cực đoan

BTL06 *[Địa lý & Bản đồ số]*

Phân tích ma trận độ cao địa hình và truy vấn vùng

Mô tả bài toán

Ma trận địa hình (DEM) lưu độ cao từng ô lưới dưới dạng ma trận phân số (độ chính xác cao). Chương trình đọc nhiều vùng địa hình, xây dựng prefix sum 2D, trả lời truy vấn tổng/trung bình vùng và phát hiện hàng/cột bất thường.

Thiết kế Struct lồng nhau

```
struct Fraction  { int num, den; }  // độ cao dạng phân số
struct DEM       { string region; int rows, cols; vector<vector<Fraction>> grid;
}
struct GeoData   { string name; vector<DEM> regions; }
→ GeoData → DEM → vector<vector<Fraction>>: struct lồng nhau + ma trận phân số
```

Input / Output

- Input: dem.csv (region, rows, cols, rồi từng dòng giá trị dạng num/den); queries.txt
- Output: query_results.txt (tổng/tb từng vùng), terrain_report.txt, anomaly_rows.txt

Yêu cầu kỹ thuật

Kỹ thuật	Yêu cầu cụ thể
Struct lồng nhau	GeoData chứa vector<DEM>; DEM chứa vector<vector<Fraction>> (ma trận phân số)
Phân số trong ma trận	Mỗi ô grid là Fraction; cộng phân số khi tính tổng hàng/cột/vùng; chuyển double khi xuất
Prefix sum 2D	Xây dựng pre[i][j] trên ma trận double (convert từ Fraction); trả lời q truy vấn O(1)
Ma trận	Tổng/tb từng hàng, từng cột; kiểm tra đối xứng; tìm hàng/cột cực đại/cực tiểu
Mảng con – Kadane trên hàng	Mảng tổng hàng → Kadane: tìm đoạn hàng liên tiếp có tổng cao nhất
Dãy con – tăng/giảm	Tìm chuỗi hàng liên tiếp có tổng hàng tăng dần (địa hình dốc lên)
Thống kê	mean/std/min/max toàn ma trận; phân bố độ cao theo nhóm (thấp/trung/cao)
File/CSV	Đọc ma trận từ CSV, validate kích thước đồng đều, xử lý missing → Fraction(0,1)
M0 + Test	≥3 bộ test; case biên: ma trận 1×1, query ra ngoài biên, toàn giá trị âm

BTL07 *[Thương mại điện tử & Logistics]*

Hệ thống phân tích đơn hàng và tối ưu lô hàng

Mô tả bài toán

Mỗi đơn hàng chứa nhiều dòng sản phẩm (line items). Chương trình quản lý đơn hàng theo ngày, tính doanh thu, tìm lô hàng tốt nhất và cửa sổ thời gian đạt mục tiêu doanh thu.

Thiết kế Struct lồng nhau

```
struct LineItem  { string product; int qty; double price; double revenue(); }
struct Order     { string id, date, customer; vector<LineItem> items; double
total(); }
struct DailySummary { string date; vector<Order> orders; double dayRevenue(); }
→ DailySummary → Order → LineItem: struct lồng nhau cấp 3
```

Input / Output

- Input: orders.csv (order_id, date, customer, product, qty, price); target.txt (S)
- Output: revenue_report.csv, best_period.txt (Kadane), window.txt (sliding window)

Yêu cầu kỹ thuật

Kỹ thuật	Yêu cầu cụ thể
Struct lồng nhau	DailySummary chứa vector<Order>; Order chứa vector<LineItem>; total() tính từ items[]
File/CSV	Đọc và nhóm LineItem vào đúng Order theo order_id; validate qty>0, price>0
Dãy con – Kadane	Trên mảng dayRevenue[]: tìm đoạn ngày liên tiếp có tổng doanh thu lớn nhất
Dãy con – tăng dài nhất	Tìm chuỗi ngày doanh thu tăng liên tiếp dài nhất (xu hướng tăng trưởng)
Sliding window	Tìm cửa sổ ngày ngắn nhất có tổng doanh thu ≥ S (dãy không âm)
Prefix sum 1D	Prefix sum trên dayRevenue[]: truy vấn tổng doanh thu đoạn [l,r] O(1)
Xử lý chuỗi	Chuẩn hoá tên sản phẩm (lowercase, trim); phân nhóm hàng hoá theo chuỗi loại
Thống kê	mean/std/min/max doanh thu ngày; chuẩn hoá Min-Max; top-k sản phẩm theo doanh thu
M0 + Test	≥3 bộ test; case biên: 1 ngày 1 đơn, doanh thu âm (hoàn trả), S không đạt được

BTL08 *[Âm nhạc & Xử lý tín hiệu số]*

Phân tích tín hiệu âm thanh và xấp xỉ đa thức hệ số phân số

Mô tả bài toán

Tín hiệu âm thanh số gồm nhiều kênh (channel), mỗi kênh là chuỗi mẫu. Chương trình đọc tín hiệu đa kênh, tính năng lượng theo cửa sổ, tìm đoạn cực đại và xấp xỉ kênh bằng đa thức hệ số phân số.

Thiết kế Struct lồng nhau

```
struct Fraction  { int num, den; }  // hệ số đa thức xấp xỉ
struct Channel   { string name; vector<double> samples; Polynomial approx; }
struct Polynomial{ int degree; vector<Fraction> coeffs; }
struct AudioFile { string filename; int sample_rate; vector<Channel> channels; }
→ AudioFile → Channel → samples[] và Channel → Polynomial → vector<Fraction>
```

Input / Output

- Input: audio.csv (channel, sample_index, value); config.txt (window_size k, poly_degree d)
- Output: energy_report.csv (năng lượng cửa sổ), best_segment.txt, poly_approx.csv

Yêu cầu kỹ thuật

Kỹ thuật	Yêu cầu cụ thể
Struct lồng nhau	AudioFile chứa vector<Channel>; Channel chứa vector<double> samples và Polynomial approx
Phân số trong Polynomial	Polynomial chứa vector<Fraction> (hệ số chính xác); tính P(x) bằng Horner; đạo hàm P'(x)
File/CSV	Đọc và nhóm mẫu vào đúng Channel; validate sample_rate > 0, xử lý missing → 0
Dãy con – Kadane	Trên mảng năng lượng cửa sổ: tìm đoạn mẫu có tổng năng lượng lớn nhất
Dãy con – tăng dài nhất	Tìm chuỗi mẫu liên tiếp tăng dần (phát hiện đoạn crescendo)
Sliding window – rolling energy	Cửa sổ k mẫu: rolling sum of squares (năng lượng); rolling mean biên độ
Prefix sum 1D	Prefix sum bình phương mẫu: truy vấn năng lượng đoạn [l,r] O(1)
Thống kê	mean/std/min/max biên độ theo từng Channel; chuẩn hoá Z-score
M0 + Test	≥3 bộ test; case biên: kênh 0 mẫu, tín hiệu toàn 0, k > n, bậc đa thức 0